

নাগরিক সেবা

Nagorik Sheba

Weekly Progress Summary Report

Weeks 1 – 3 | Requirements → Design → Sprint 1 Implementation

Project	Nagorik Sheba — Agentic Civic Complaint Platform for Bangladesh
Document	Weekly Progress Summary (covers SRS v0.1, v0.2, v0.3)
Course	Web Programming and Mobile Application Development
Institution	CSE Discipline, Khulna University, Khulna
Supervisor	Dr. Kazi Masudul Alam, Professor, CSE Discipline, Khulna University
Prepared by	Mozaza Al Zaman (220210) & Md. Tawfiqul Islam (220214)
Date	August 2026
Coverage	Week 1 (Requirements) · Week 2 (Design) · Weeks 3 (Sprint 1 Development)

Overview — Progress at a Glance

This report summarizes the work completed across the first four weeks of the Nagorik Sheba project, drawn from SRS v0.1 (Week 1), SRS v0.2 (Week 2), and SRS v0.3 (Weeks 3–4). Each section below covers what was planned, what was delivered, tools used, and the status at the end of that period.

Period	Phase	Primary Output	Status
Week 1	Requirements Gathering	SRS v0.1 — scope, objectives, functional & non-functional requirements	 Complete
Week 2	Design & Planning	SRS v0.2 — wireframes, ER diagram, flowchart, GitHub board	 Complete
Weeks 3	Development Sprint 1	SRS v0.3 — working auth, AI pipeline, web Computer + mobile both platform, backend API	 Complete

Week 1 — Requirements Gathering & Initial SRS

WEEK 1	Requirements Phase	COMPLETE
--------	--------------------	----------

1.1 Goal for the Week

Establish the problem definition, scope, and full set of functional and non-functional requirements for Nagorik Sheba, and produce SRS v0.1 as the foundational course deliverable.

1.2 What Was Accomplished

- Defined the project purpose and 8 core objectives — including automated complaint routing, authenticity verification, urgency-weighted prioritization, multi-channel access (web/mobile/SMS), transparency features, resilient AI agent design with rule-based fallbacks, and Bangladesh-specific three-tier local government modeling
- Scoped the platform to cover 7 complaint categories (road, water, electricity, gas, sanitation, streetlight, drainage) plus a public toilet locator, emergency directory, complaint search/endorsement, and public transparency scorecards
- Documented product perspective, functions summary, and 4 user classes — Citizen, Department Staff, Administrator, and unauthenticated Public Visitor
- Specified the operating environment — Node.js/Express server, SQLite prototype with a PostgreSQL/PostGIS upgrade path, mobile-first responsive client, and SMS aggregator integration
- Wrote out design constraints, most notably the NID verification approach: since real Bangladesh Election Commission NID database access is restricted to licensed institutions, the project uses a synthetic "SPECIMEN" watermarked NID dataset paired with team members' own consenting selfies for a perceptual-hash self-consistency check — no real third-party identity data is collected or stored
- Authored functional requirements across 8 modules — User Registration & Authentication, Complaint Submission/Search/Endorsement, AI Agent Pipeline, Department Staff Dashboard, Public Toilet Locator & Rating, Emergency (Fire Service) Directory, SMS Channel, and Public Transparency & Participation — each requirement tagged with a REQ-XX-## ID and priority level
- Defined external interface requirements (user, hardware, software, communication interfaces) and non-functional requirements across performance, security, data privacy, usability, reliability, scalability, availability, and accessibility
- Proposed a 4-layer system architecture — Client Layer, Application/Agent Layer (Moderation → Classification → Verification → Priority → Routing agents, each with a rule-based fallback), Data Layer, and a Bangladesh-specific Jurisdiction Layer using the Barikoi API for ward/zone resolution
- Compiled Appendix A (an 18-term glossary) and Appendix C (an issues list tracking 9 open items for resolution in later weeks)

1.3 Tools Used

AI-assisted requirement drafting, reviewed and edited by the project team; Microsoft Word for document authoring.

1.4 Open Issues Carried Forward

- Barikoi ward-level API coverage confirmed for Dhaka but not yet verified for Khulna (the project's demo city)
- Whether a native mobile app is a hard course requirement, or a responsive PWA suffices, needed supervisor clarification
- Production SMS aggregator not yet selected (Twilio confirmed only as prototype stand-in)
- Public toilet and emergency-service seed data not yet collected
- ER diagram, system architecture diagram, and data flow diagram flagged as not yet finalized — explicitly scheduled for Week 2

1.5 Status at Week 1 Close

SRS v0.1 completed and submitted as the Week 1 draft, with all analysis-model diagrams and several data-collection items explicitly deferred to Week 2 rather than omitted from scope.

Week 2 — Design & Planning

WEEK 2	Design Phase	COMPLETE
--------	--------------	----------

2.1 Goal for the Week

Close the design gaps identified at the end of Week 1 — turn the textual requirements from SRS v0.1 into concrete visual design artifacts (wireframes, database design, architecture diagrams) and set up the team's project-management workflow ahead of development.

2.2 What Was Accomplished

- Designed complete UI/UX wireframes in Figma — 8 citizen mobile app screens (Login, Home Feed, Submit Complaint, My Complaints, Complaint Detail, Nearby Facilities, Notifications, Profile) and 5 government staff dashboard screens (Login, Overview, Complaint Detail, Live Map, Analytics)
- Produced the full Entity-Relationship diagram — 8 database tables (USER, LGI, STAFF, COMPLAINT, COMPLAINT_VOTE, COMPLAINT_TRAIL, PUBLIC_FACILITY, NOTIFICATION) using draw.io and Mermaid, resolving the ERD gap flagged in Week 1
- Documented key schema design decisions — a self-referencing foreign key on COMPLAINT for duplicate-merge handling, a nullable staff_id on COMPLAINT_TRAIL to distinguish AI-agent entries from human staff actions, and a bounding-box representation for LGI jurisdiction boundaries
- Produced the system flowchart for the 5-agent AI complaint pipeline (Photo Verifier → Classifier → Duplicate Checker → Priority Ranker → LGI Router), replacing the Week 1 placeholder architecture diagrams
- Refined the use case diagram covering the three primary actors — Citizen, Government Staff, and Administrator
- Configured the GitHub Projects board — a 5-column Kanban workflow (Backlog, To Do, In Progress, In Review, Done), 9 labels, 2 milestones (Sprint 1 and Sprint 2), and a branch strategy (protected main, per-member dev branches, mandatory PR review)
- Allocated Sprint 1 and Sprint 2 tasks between the two team members and documented the split in the SRS

2.3 Tools Used

Figma with AI-assisted design tools (wireframes), draw.io and Mermaid (ER diagram and flowcharts), GitHub Projects (task board and milestones).

2.4 Status at Week 2 Close

SRS v0.2 completed with all Week 2 design deliverables finalized — UI wireframes, ER diagram, system flowchart, refined use case diagram, and a fully configured GitHub Projects board ready to drive Sprint 1 development.

Week 3 — Development Sprint 1

WEEKS 3–4	Development Phase	COMPLETE
-----------	-------------------	----------

3.1 Goal for the Sprint

Turn the Week 2 design artifacts into a working, end-to-end system across backend, web, and mobile — implementing authentication, the full 5-agent AI pipeline, and the core citizen and staff-facing screens designed in Week 2.

3.2 What Was Accomplished

Authentication

- Built face-verified citizen registration using face-api.js + TensorFlow WASM, comparing a live selfie against the NID card photo with up to 3 retry attempts for low-quality images
- Implemented phone OTP and email/password login for citizens, with pre-seeded staff accounts and JWT-based role routing

AI Pipeline

- All 5 agents made fully operational and running server-side in Node.js, each with a rule-based implementation and a Claude API fallback
- Classifier agent validated at 12/12 passing test cases across Bangla-script and Roman-Bangla keyword inputs
- Duplicate Checker implemented with a 250-metre GPS radius scan and vote-merging on match

Location & Routing

- Two location-entry methods built: live GPS pin-drop with OpenStreetMap Nominatim reverse geocoding, and a custom Division → District → Office Type → Office Name dropdown picker
- LGI routing seeded with 4,880 real local government offices (12 City Corporations, 328 Pourashavas, 4,540 Union Parishads)

Web Frontend (React 18)

- Built and connected all core screens — Explore/Landing feed, Report Issue (two-mode location picker, photo + voice input), Track Complaint (status timeline + voting), Emergency Directory, Staff Dashboard (priority queue + ETA), and Notifications
- Full Bangla/English language toggle implemented across the UI, resolving the Week 1 usability requirement

Backend & Database

- 14+ REST API endpoints implemented (auth, complaints, voting, staff queue, emergency lookup, notifications, LGI resolution)
- SQLite database fully created and seeded — 9 tables, 4,880 LGI offices, and 355 emergency service records
- 5 notable bugs identified and fixed during the sprint, including missing staff accounts for Union Parishad offices and a mobile photo-upload MIME-type rejection issue

Process

- GitHub Copilot used throughout for route boilerplate, database models, and widget scaffolding
- Unit tests generated for the AI pipeline and authentication endpoints, all passing
- All Sprint 1 features developed on personal branches and merged to main via peer-reviewed Pull Requests, per the Week 2 branch strategy

3.3 Tools Used

React 18 + Vite + Tailwind CSS, Node.js/Express, SQLite, Flutter/Dart, face-api.js + TensorFlow WASM, OpenStreetMap + Nominatim, GitHub Copilot, GitHub Pull Requests.

Cumulative Summary — Weeks 1 to 3

The table below tracks how each major requirement or design item introduced in Week 1 progressed through design and into a working implementation.

Area	Week 1 (v0.1)	Week 2 (v0.2)	Weeks 3 (v0.3)
Requirements	Drafted — 8 modules, REQ-XX-## IDs	—	—
UI Design	Not started	Wireframed — 13 screens	Built — React 18 + Flutter
Database	Conceptual only	ER diagram — 8 tables	Implemented & seeded — 9 tables
AI Agent Pipeline	Specified — 5 agents planned	Flowcharted	Operational — 12/12 tests pass
Auth / NID Check	Sandbox perceptual-hash approach	Login screens wireframed	Face-api.js verification live
Jurisdiction Routing	Barikoi API planned (Khulna unverified)	GPS bounding-box design	Nominatim geocoding implemented
Project Management	—	GitHub Projects board configured	76 tasks tracked, PR workflow active
Test case	—	—	Test case generation and verified. All the test case passed/